CMTS: Consistency Model for Time Series Forecasting

Abstract

Diffusion models have recently showcased their applicability to time series forecasting (TSF). Generally, in most methods, diffusion-based TSF can be formulated as a conditional diffusion process, where we predict the future time series from noise, conditioned on the historical time series. Despite the inherent capability of diffusion models to capture uncertainty in TSF, one of their main disadvantages, inherited from their image generation baselines, lies in prohibitively slow inference. Various methods have been introduced to address this issue by proposing faster inference algorithms. One notable method are consistency models (CM), which have showcased their capability of generating images in as little as one step. In this work, we develop a **C**onsistency **M**odel for **T**ime **S**eries Forecasting, demonstrating the applicability of CMs to the task of TSF. On top of replacing the diffusion denoiser with a consistency one, we also introduce a modified consistency objective that also applies certain degradations to the conditions to enhance learning capability. Our model achieves the best results in terms of MSE and MAE metrics on Exchange and Stock datasets among diffusion-based models, while being competitive across four other datasets. Additionally, CMTS is capable of achieving this in only one sampling step, further enhancing its utility.

1 Introduction

In the task of time series forecasting, we are provided with some historical data points, with each point accompanied by the time the data point was observed. The goal is to predict future data points, based on the historical points provided. Time series prediction has a wide range of applications, including finance, healthcare, and weather forecasting.

Diffusion models [1, 2], best known for their remarkable performance in computer vision tasks such as image generation, have recently emerged as a promising framework for time series applications [3, 4, 5, 6, 7, 8, 9, 10, 11]. Diffusion models add noise to some clean data and aim to learn to iteratively remove the noise to obtain the reconstruction of the data. This iterative procedure results in slow inference, which is unacceptable in many real-time applications. To address this issue, various faster-inference alternatives have been proposed [12, 13, 14, 15, 16, 17]. One of the most prominent families of fast diffusion-based models are consistency models [15], which have shown their ability to generate high-quality samples in a few steps, including one step. In this project, we will develop a **C**onsistency **M**odel for **T**ime **S**eries forecasting (CMTS). The contributions of CMTS include the following:

- Inspired by the denoising process of ARMD, we develop CMTS, a fast-diffusion-sampling consistency-based model for TSF

Submitted to 38th Conference on Neural Information Processing Systems (NeurIPS 2024). Do not distribute.

- On top of supplanting the diffusion denoiser with a consistency one, we introduce a novel consistency condition, which involves further degrading the student input (historical time series), where we randomly choose several points and replace them with linear interpolations of their immediate neighbors
- CMTS achieves best results in terms of MSE and MAE metrics on two datasets, and competitive results on four others

A high-level overview of CMTS is shown in Fig. 1

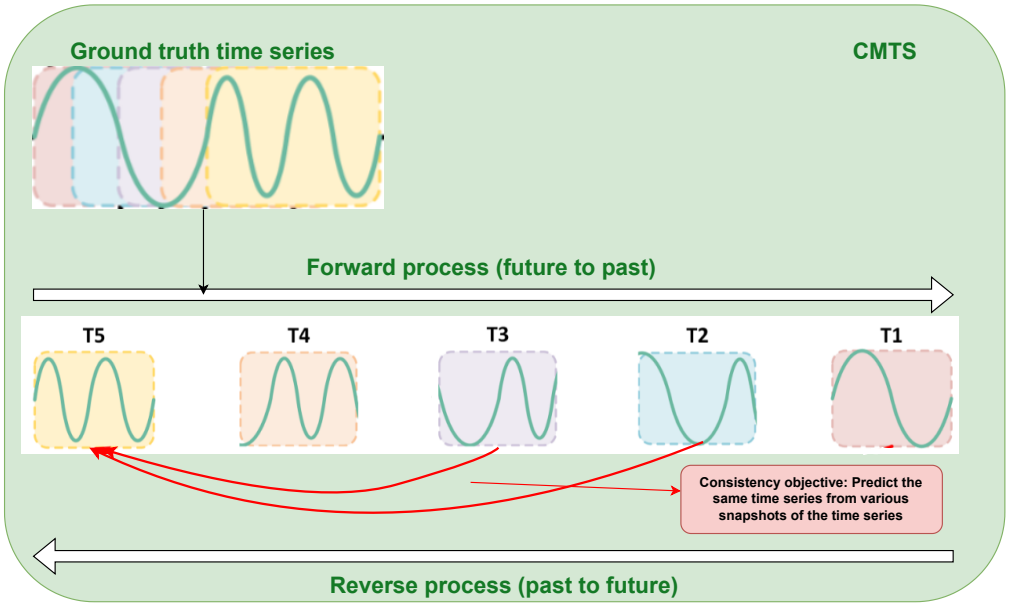

Figure 1: High-level overview of CMTS. A ground-truth time series can be split into several snapshots using a sliding window operation. The consistency objective consists of predicting the time window furthest in the future (labeled as $T5$ in the figure from any other snapshot). Similarly to ARMD, the "diffusion" process consists of a deterministic sliding window operation.

2 Related Work

2.1 Diffusion models for TSF

Diffusion models (DMs) have proven to excel in tasks such as image generation. More recently, DMs have been applied to the task of TSF due to their ability to capture uncertainty associated with this task. TimeGrad [3] is an autoregressive model which applies conditional diffusion models to multivariate time series forecasting, where the generation of future data points is conditioned on the past ones. CSDI [4] is a non-autoregressive variant also suited for time series imputation, where we have missing data points at some specific time steps. SSSD [5] replaces the transformer-based architecture of TimeGrad and CSDI with a structured space model. TimeDiff [6] enhances the learning process by incorporating random parts of ground-truth future to aid conditioning. DiffusionTS [7] systematically decomposes the time series signal into trend, season, and error; further enhancing the learning process. TMDM [8] incorporates a transformer-based encoder to extract relevant features from historical data, which is used to generate the condition to predict future steps. RATD [9] employs embedding-based retrieval to obtain reference predictions, which are used as an additional condition to generate future timesteps. ARMD [10] is a non-denoising alternative which draws a correspondence between diffusion and forecasting timesteps by utilizing a sliding window to predict future points. MG-TSD [18] and mrDiff [19] employ an iterative coarse-to-fine generation procedure, where they first apply the diffusion process to generate coarse trends. Then, the coarse trend is used as a reference to generate finer trends until the final time series prediction is obtained. NSDiff [11]

departs from the standard Gaussian prior assumption and models a dynamic (data-dependent) prior, which forms both the mean and the variance as a function of the data.

2.2 Consistency models

Although DMs have achieved impressive results in image generation, and various other tasks, including TSF, they still suffer from slow inference. This slow inference is caused by the iterative denoising algorithm, where the original methods [2, 1] generated samples in up to 1000 steps. This can be prohibitively computationally expensive, especially in real-time setting. An ongoing research direction, which has made significant headway in the past couple of years, consists of speeding up diffusion inference. DDIM [20] addresses this by skipping sampling steps, speeding up the inference process up to 50 times. Distillation-based methods, such as Progressive Distillation (PD) [14] first train the model for the higher number of inference steps, and halve it iteratively as the training progresses, leading to few-step diffusion sampling. EDM [13] additionally analyzes the design space of diffusion models, further enabling few-step high-quality sampling. EDM can be said to be a precursor of one of the most prominent fast-inference diffusion baselines are Consistency models (CMs) [15], which we will explain in more detail.

CMs employ a student-teacher-based learning framework to enable few-step sampling. CMs are naturally distillation-based, which means that they perform best when initialized from a pre-trained diffusion model, which can be costly to train. iCT [21] proposes a set of design choices which enable high-quality generation when the CM is trained without the aforementioned initialization. Over the past several years, various extensions of CMs have been proposed. Latent consistency models (LCM) [22] have extended CMs to high-quality text-to-image generation. Consistency Trajectory Models (CTM) [23] predict the entire generation trajectory instead of just the endpoint like [15] and [21]. Easy consistency models (ECM) [24] suggest a hybrid diffusion-consistency method which achieves impressive results in image generation with much less training compared to the pioneering baselines. Consistency models have also been recently combined with flow-based models [25], resulting in even more powerful baselines that use the best of both worlds. Some prominent examples of these are Shortcut Models [26] and MeanFlow [17]. To the best of our knowledge, consistency models have not yet been applied to time series forecasting. Although we have listed various advanced baselines, we will use the baseline from [21] in our method.

3 Method

3.1 Preliminaries - consistency models

As was mentioned in Section 2.2, CMs utilize a student-teacher training paradigm. In particular, in [15], during training, we add two noise levels to clean data - the *student* one, σ_s, which has higher variance, and the *teacher* one, σ_t, which has lower variance. Generally, consistency models are optimized for the following loss:

$$\mathcal{L}_{consistency} = d(f_\theta(x_{\sigma_s}, \sigma_s), f_{\theta^-}(x_{\sigma_t}, \sigma_t)), \tag{1}$$

where x_{σ_s} denotes noisy data resulting from adding noise of variance σ_s to clean data x_0. Additionally θ denotes the weights of the student (online) network, and θ^- the weights of the teacher (offline) network. θ^- is updated using the exponential moving average (EMA) of θ. We can observe from Eq. 1 that the optimization process consists of comparing two *estimates* of the ground truth. This can easily lead to collapsing to a trivial solution which is far away from the ground truth. To prevent this, CMs also employ the following boundary condition:

$$f_\theta(x_{\sigma_{min}}, \sigma_{min}) = x_{\sigma_{min}}, \tag{2}$$

which enforces mapping data with a very small amount of noise (σ_{min}) added to itself. Broadly speaking, there are two ways to train consistency models: *Consistency distillation (CD)* and *Consistency Training (CT)*. In the former, both the student and the teacher networks are initialized with a pretrained diffusion model (in particular, [15] initializes them with a pretrained EDM [13]). Additionally, x_{σ_t} is obtained by applying an appropriate number of sampling steps of the pretrained diffusion models, with x_{σ_s} as the input. In [15], student and teacher are one time step apart, although this has been generalized to multiple steps [22] and even the continuous domain [23, 24, 17]. In consistency training (CT), the networks are initialized randomly, and x_{σ_t} is not obtained using a

pretrained sampler, but by adding noise σ_t directly to the clean input x_0. In our setting, we will use CT, since time series datasets we used are low-dimensional, resulting in their training being relatively quick, regardless of the method used.

3.2 Problem setting

In this work, we focus on conventional time series forecasting, where we are provided a historical time series $X_{-L+1:0}$, where L denotes the number of historical time steps. The objective is to predict the future time series denoted $X_{1:T}$ where T denotes the number of future time steps. In our setting, we will focus on multi-variate time series, where each X_t is a vector of features, whose number depends on the dataset.

3.3 CMTS

3.3.1 Forward process

Our method is primarily based on ARMD [10], which uses a non-standard diffusion process. Most other works, such as TimeGrad [3], CSDI [4] etc. model the diffusion process as a standard denoising process, where we add Gaussian noise (forward process), and the model learns to remove the noise (reverse process). In particular, in these settings, the historical input $X_{-L+1:0}$ is used as a condition to generate $X_{1:T}$, starting from standard Gaussian noise of appropriate dimensions. In the case of ARMD, however, the initial input is the clean future time series. The forward process consists of moving the time window towards the historical time series, where the number of sampling steps corresponds to the number of historical time series steps. More precisely, one forward diffusion process would correspond to moving one step backwards in the time series. Analogously, one reverse diffusion process would correspond to moving towards the future. More formally, $X^0_{1:T}$ serves as the initial input, where the superscript denotes the diffusion time step, and the subscript denotes the time window. Each intermediate state (diffusion time step $t \in [0, T)$) can be denoted as $X^t_{1-t:T-t}$. The historical time series, which denotes the last forward diffusion step, can be denoted as $X^T_{-T+1:0}$. A single step of the forward process can be formally denoted as:

$$X^t_{1-t:T-t} = Slide(X^{t-1}_{2-t:T-t+1}, 1), \tag{3}$$

where $Slide(X, k)$ denotes moving k steps to the past. This can be generalized to t time steps as follows:

$$X^t_{1-t:T-t} = Slide(X^0_{1:T}, t) \tag{4}$$

Similarly to the DDPM [1] formulation of the Gaussian denoising process, Eq. 4 can be decomposed as:

$$Slide(X^0_{1:T}, t) = \sqrt{\overline{\alpha}_t} X^0_{1:T} + \sqrt{1 - \overline{\alpha}_t} z^t, \tag{5}$$

where the meaning of $\overline{\alpha_t}$ is similar to the one in DDPM [1]. The term z^t can be said to denote the *evolution trend*, and serves a purpose similar to Gaussian noise in standard diffusion models for time series. However, the key difference is that z^t is deterministic (unlike Gaussian noise, which is stochastic), and it can be directly derived from Eq. 4

3.3.2 Reverse process

In the reverse process, the goal is to predict $X^0_{1:T}$ from $X^t_{1-t:T-t}$. At inference time, $t = T$ (i.e. no access to any ground point future time step). Since the time series data used in this work are low-dimensional (5-10 features per time step), the model first applies a linear module to predict the distance from $X^t_{1-t:T-t}$ to $X^0_{1:T}$. Then, the time step t is applied adaptively to obtain the final estimation:

$$\hat{X}_0(X^t, t, \theta) = \frac{W(t) * X^t_{1-t:T-t} + (1 - bW(t)) * D}{(1 + cW(t))^d}, \tag{6}$$

where θ denotes the prediction network, $W(t)$ denotes a coefficient which decreases as we increase t. $W(t)$ is initialized with $\overline{\alpha}_t$, and then updated throughout training. b, c and d are considered hyperparameters.

3.3.3 Consistency training

In the previous subsections, we focused on the parts of the model that are similar to ARMD - here, we will highlight the key differences. In ARMD, the objective is to predict the trend, in a similar fashion DDPM predicts noise. In CM, however, the model learns to predict data - so, in our case, we also learn to predict data directly. However, using the result in Eq. 6 directly results in poor predictions, similarly to CM. Instead, we opt for a parameterization similar to [13]

$$\hat{X}_0(X^t, t, \theta) = c_{skip}(t)X^0_{1:T} + c_{out}(t)F(X^t, t), \tag{7}$$

where $F(X^t)$ corresponds to $\hat{X}_0(X^t, t, \theta)$ from Eq. 6. Similarly to [15], $c_{skip}(t)$ and $c_{out}(t)$ are differentiable functions for which $c_skip(0) = 1$ (to ensure the boundary condition, defined in Eq. is satisfied) and $c_{out}(0) = 0$. Since our model, like ARMD, is deterministic, we cannot directly use functions from [15]. In our case, we chose $c_{skip} = 1 - t/T$ and $c_{out} = t/T$.

Next, we need to formulate the consistency objective in our model. Given the ARMD formulation of the diffusion process, we can do so as follows:

$$\mathcal{L}_{consistency} = d(\hat{X}_0(X^t, t, \theta), \hat{X}_0(X^{t-1}, t-1, \theta^-)), \tag{8}$$

where the first argument of $d(\cdot, \cdot)$ denotes the student input, and the second one denotes the teacher input. We point out that the teacher network is still updated as the EMA of the student network θ. In our case, the L_2 loss was the best choice for the distance metric d.

Enhanced consistency loss. The loss function obtained in Eq. 8 is a relatively trivial extension of the diffusion loss from ARMD. One way in which we can extend our framework would be to not only make student one step further in the past, but to also corrupt the student input itself further. We opted for a straightforward way to do this. Namely, the student input X^t, in its original form, contains all time steps during training. However, we opted for removing K points (out of T) and replacing them with linear interpolations of the adjacent points. Here are additional considerations we adhered to:

- In order to not overly corrupt the student input and make learning unnecessarrily difficult, we set $K << T$.
- We also ensure that we don't sample two adjacent points, i.e. that for any removed point, the immediate neighbors are still available (i.e. not among the removed points)
- Since removing points closer to the future time steps may compromise generation quality (especially if we remove the first point before the future time series), we sample the K time steps to be removed from a multinomial distribution, where the probability of sampling each of the T points decreases as the points get closer to the future.

We apply this degradations only to the student input. The intuition behind this is that the student should also learn to achieve prediction capabilities equal to (or better than) the teacher ones. This intuition was formalized through an additional GAN-based loss term in [23], which we will not consider in this work. The new and final consistency loss is now:

$$\mathcal{L}_{consistency} = d(\hat{X}_0(X'^t, t, \theta), \hat{X}_0(X^{t-1}, t-1, \theta^-)), \tag{9}$$

where X'^t is the student input with the aforementioned degradations applied.

3.3.4 Inference

Although consistency models are trained for single-step inference, they are also capable of sampling in more than one step. This is possible through an alternating noising-denoising (or in our case sliding backward-forward in time) process. Namely, we start with the historical time series $X^T_{-L+1:0}$ and first predict the entire future time series $X^0_{1:T}$. Then, at every subsequent step, we noise and predict another instance of $X^0_{1:T}$. However, the sliding step is decreased at every subsequent step. If we sample for N timesteps, the length by which we move the sliding window at time step n is:

$$(1 - \frac{n}{N})L. \tag{10}$$

However, this iterative sampling procedure actually gave worse results in our case, so we will report only one-step sampling results in Sec. 4.

4 Experiments

4.1 Datasets

We ran our method on the following datasets: ETTh1, ETTh2, ETTm1, ETTm2, Exchange and Stock datasets. For all of these datasets, both the historical sequence length and the future time series length is 96. For the ETT datasets, the feature length per time step is 7. For Exchange, the feature length is 8, and for Stock it is 5.

4.2 Baselines and metrics

We compared our results with ARMD [10], and also some preceding diffusion-based baselines ARMD compared their results to, such as: TimeGrad [3], D^3VAE [27], TSDiff [28], MG-TSD [18] and Diffusion_TS [7].

We evaluated our method on the mean square error (MSE) and mean absolute error (MAE) metrics.

4.3 Main results

In this section, we will report the main results on the time series forecasting (TSF) task, where we used MSE and MAE metrics. The results in terms of MAE are reported in Table 1, and in terms of MSE in Table 4.3. In both tables, NFE denotes the number of sampling timesteps, which is the convention borrowed from [15]. We can first observe that our method provides the best results on Exchange and Stock datasets, for both MSE and MAE. Additionally, it is the runner-up on ETTh1 and ETTm1 datasets both in terms of MSE and MAE. Our method does the least well on ETTh2 and ETTm2 datasets across both metrics. However, one aspect in which our model does excel is the inference speed. Namely, unlike all the other baselines which need to use at least 20 sampling steps to achieve good results (with TSDiff being the runner-up), our method is capable of achieving good results in a single step. This is one of the main motivations to introduce CM-based method in the first place. Our best version involves interpolating either $K = 2$ or $K = 4$ timesteps (depending on the dataset) with their immediate neighbors. We discuss the choice of this parameter in Sec. 4.4.

Table 1: MSE results. The best method is denoted in boldface and the runner-up is underlined.

Method	NFE	ETTh1	ETTh2	ETTm1	ETTm2	Exchange	Stock
TimeGrad	24	0.884	0.297	0.661	0.182	0.508	0.333
D3VAE	100	1.123	0.389	0.644	0.394	0.240	0.345
TSDiff	20	0.614	0.470	0.686	0.242	0.125	0.330
MG-TSD	60	1.096	**0.295**	0.690	0.202	0.396	0.365
Diffusion-TS	50	0.643	0.544	0.678	0.497	0.275	0.416
ARMD	96	**0.445**	0.311	**0.337**	**0.181**	0.093	0.235
CMTS (ours)	**1**	0.505	0.630	0.358	0.545	**0.079**	**0.079**

Table 2: MAE results. The best method is denoted in boldface and the runner-up is underlined.

Method	NFE	ETTh1	ETTh2	ETTm1	ETTm2	Exchange	Stock
TimeGrad	24	0.884	**0.297**	0.661	**0.254**	0.508	0.333
D3VAE	100	0.728	0.373	0.538	0.410	0.371	0.390
TSDiff	20	0.521	0.418	0.603	0.311	0.240	0.365
MG-TSD	60	0.765	0.345	0.631	0.278	0.460	0.453
Diffusion-TS	50	0.586	0.494	0.613	0.459	0.382	0.533
ARMD	96	**0.459**	0.338	**0.376**	0.255	0.203	0.269
CMTS (ours)	**1**	0.495	0.566	0.412	0.520	**0.182**	**0.193**

4.4 Ablation study on degradation by interpolation

In this section, we will analyze the utility of each component of our method. On top of comparing various versions of our method with each other, we have also included results for ARMD, as the closest relative of CMTS. We tested the following values of $K = 0, 1, 2, 4, 8, 16$, where $K = 0$ denotes the standard case without any interpolation. The results in terms of MSE are shown in Tab. 3, and in terms of MAE in Tab. 4. We generally notice that both MSE and MAE plateau at either $K = 2$ or $K = 4$, depending on the dataset, although it can be argued that for $K = 0$ and $K = 1$ the model provides good results. We also note that, as we increase K above these optimal values, both metrics start to degrade significantly. This corresponds to our intuition that interpolating too many points can overly corrupt our condition, hindering the learning process.

Table 3: Ablation study - MSE metric. CMTS result for the best K is denoted in boldface

Method	K	ETTh1	ETTh2	ETTm1	ETTm2	Exchange	Stock
ARMD	N/A	0.445	0.311	0.337	0.181	0.093	0.235
CMTS (ours)	0	0.527	0.648	0.378	0.558	0.088	0.085
CMTS (ours)	1	0.519	0.638	0.364	0.550	0.086	0.083
CMTS (ours)	2	**0.505**	**0.630**	**0.358**	**0.545**	0.082	0.080
CMTS (ours)	4	0.509	0.641	0.360	0.549	**0.079**	**0.079**
CMTS (ours)	8	0.580	0.699	0.387	0.573	0.084	0.082
CMTS (ours)	16	0.631	0.778	0.413	0.612	0.091	0.090

Table 4: Ablation study - MAE metric. CMTS result for the best K is denoted in boldface

Method	K	ETTh1	ETTh2	ETTm1	ETTm2	Exchange	Stock
ARMD	N/A	0.459	0.338	0.376	0.255	0.203	0.269
CMTS (ours)	0	0.501	0.576	0.420	0.531	0.195	0.201
CMTS (ours)	1	0.498	0.570	0.417	0.525	0.189	0.199
CMTS (ours)	2	**0.495**	**0.566**	**0.412**	**0.520**	0.184	0.194
CMTS (ours)	4	0.497	0.569	0.414	0.521	**0.182**	**0.193**
CMTS (ours)	8	0.553	0.620	0.444	0.570	0.191	0.204
CMTS (ours)	16	0.600	0.682	0.486	0.633	0.210	0.222

5 Discussion

In this work, we introduce CMTS, a consistency-based model for TSF. To the best of our knowledge, CMs have not been applied to this task. CMTS demonstrates the capability to generate accurate time series predictions, which is showcased in achieving the best MSE and MAE across two datasets. Additionally, introducing the enhanced consistency objective was also proven beneficial, as was shown in Sec. 4.4

6 Limitations and future work

Although CMTS does achieve good results on TSF, there are some apparent limitations. One of them is that it was not able to surpass ARMD (which it was based upon) on 4 out of 6 datasets we experimented on. One future direction to examine would be to try sampling with more than one time step. However, the procedure described in 3.3.4 did not scale well to TSF, so we may need to seek alternative formulations. Another somewhat orthogonal direction, would be to explore more advanced consistency (or related) baselines, such as CTM [23], ECM [24] or flow-based models [25, 17]. Another aspect, which is crucial in TSF, and which CMTS fails to capture, is the prediction uncertainty. Most methods introduce some sort of noise in their prediction, which neither ARMD or CMTS do. So, measuring uncertainty through standard TSF metrics such as CRPS or QICE is not feasible here. An additional future direction would be to enhance CMTS so that it captures uncertainty, for example, through Gaussian noise addition.

References

[1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[2] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.

[3] Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *International conference on machine learning*, pages 8857–8868. PMLR, 2021.

[4] Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. Csdi: Conditional score-based diffusion models for probabilistic time series imputation. *Advances in neural information processing systems*, 34:24804–24816, 2021.

[5] Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.

[6] Lifeng Shen and James Kwok. Non-autoregressive conditional diffusion models for time series prediction. In *International Conference on Machine Learning*, pages 31016–31029. PMLR, 2023.

[7] Xinyu Yuan and Yan Qiao. Diffusion-ts: Interpretable diffusion for general time series generation. In *The Twelfth International Conference on Learning Representations*.

[8] Yuxin Li, Wenchao Chen, Xinyue Hu, Bo Chen, Baolin Sun, and Mingyuan Zhou. Transformer-modulated diffusion models for probabilistic multivariate time series forecasting. In *The Twelfth International Conference on Learning Representations*, 2024.

[9] Jingwei Liu, Ling Yang, Hongyan Li, and Shenda Hong. Retrieval-augmented diffusion models for time series forecasting. *Advances in Neural Information Processing Systems*, 37:2766–2786, 2024.

[10] Jiaxin Gao, Qinglong Cao, and Yuntian Chen. Auto-regressive moving diffusion models for time series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 16727–16735, 2025.

[11] Weiwei Ye, Zhuopeng Xu, and Ning Gui. Non-stationary diffusion for probabilistic time series forecasting. *arXiv preprint arXiv:2505.04278*, 2025.

[12] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*.

[13] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *Advances in neural information processing systems*, 35:26565–26577, 2022.

[14] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations*.

[15] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. 2023.

[16] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. In *The Thirteenth International Conference on Learning Representations*.

[17] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. *arXiv preprint arXiv:2505.13447*, 2025.

[18] Xinyao Fan, Yueying Wu, Chang Xu, Yuhao Huang, Weiqing Liu, and Jiang Bian. Mg-tsd: Multi-granularity time series diffusion models with guided learning process. *arXiv preprint arXiv:2403.05751*, 2024.

[19] Dagyeong Na and Junseok Kwon. Mrdiff: Time series anomaly detection using multi-level reconstruction diffusion. In *2024 IEEE International Conference on Data Mining Workshops (ICDMW)*, pages 688–695. IEEE, 2024.

[20] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.

[21] Yang Song and Prafulla Dhariwal. Improved techniques for training consistency models. *arXiv preprint arXiv:2310.14189*, 2023.

[22] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. *arXiv preprint arXiv:2310.04378*, 2023.

[23] Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Naoki Murata, Yuhta Takida, Toshimitsu Uesaka, Yutong He, Yuki Mitsufuji, and Stefano Ermon. Consistency trajectory models: Learning probability flow ode trajectory of diffusion. *arXiv preprint arXiv:2310.02279*, 2023.

[24] Zhengyang Geng, Ashwini Pokle, William Luo, Justin Lin, and J Zico Kolter. Consistency models made easy. *arXiv preprint arXiv:2406.14548*, 2024.

[25] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[26] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. *arXiv preprint arXiv:2410.12557*, 2024.

[27] Yan Li, Xinjiang Lu, Yaqing Wang, and Dejing Dou. Generative time series forecasting with diffusion, denoise, and disentanglement. *Advances in Neural Information Processing Systems*, 35:23009–23022, 2022.

[28] Marcel Kollovieh, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–28364, 2023.